

UR3e Web Control

A Senior Project Report

presented to
the Faculty of California Polytechnic State University
San Luis Obispo

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Computer Engineering

By
Tenzin Konchok

March 2026

Advised by Dr. Javier Gonzalez-Sanchez

Abstract

This project investigates how a Universal Robots UR3e can be controlled through a browser-based interface rather than through low-level robot commands alone. The final system supports three interaction modes: chess-style board movement and suction-based pick/place, tic-tac-toe movement and pen-based marking, and whiteboard drawing from uploaded raster images. The implementation combines a React frontend, a Node.js/Express backend, calibrated workspace mapping, URScript generation, RTDE-based live status feedback, and an OpenCV-based drawing pipeline.

The main engineering result is an integrated control platform that translates operator inputs, such as board squares, tic-tac-toe cells, and raster images, into executable robot actions. The system includes dry-run support, single-client control locking, drawing safety checks, and preview support for image-based drawing. The work is best framed as an engineering implementation that connects existing robot interfaces into one usable workflow, not as a new robotics algorithm.

Due to limited project time, development and validation were completed in URSim rather than on the physical robot. The evaluation covers the software workflow, generated URScript path, preview metrics, and simulator-side status behavior, but it does not cover physical placement accuracy, suction reliability, pen contact quality, or real hardware timing. The main limitations are calibration sensitivity, image-quality dependence in whiteboard mode, and ETA estimates that remain approximate rather than exact. Overall, the project provides a structured control platform and a practical foundation for future hardware testing.

1 Problem Statement

Universal Robots provides low-level external interfaces such as URScript, RTDE, and dashboard control, but those interfaces do not by themselves provide a complete operator workflow for structured educational and laboratory interaction modes such as board manipulation or image-driven drawing. As a result, there is a gap between raw robot communication and a usable system in which an operator can select a task, preview behavior, and execute it through a single browser-based interface.

The engineering problem addressed in this project is how to build a web control system for a Universal Robots UR3e that can translate user inputs, such as chess squares, tic-tac-toe cells, and uploaded raster images, into calibrated robot actions while maintaining safe execution flow, clear system state, and modular software structure. The system also needs to support multiple task modes within one interface and remain practical for URSim-based development before any physical deployment.

This project addresses that problem by integrating frontend interaction, backend routing, calibration-aware mapping, image vectorization, URScript generation, and live status monitoring into one control platform. The resulting system provides structured task execution rather than only low-level command dispatch.

2 Goals, Objectives, and Scope

The formal goal of this project was to develop and validate, in URSim, a web-based control platform for the Universal Robots UR3e that converts user inputs into calibrated robot actions. The system was intended to support board movement, suction-based pick/place, pen-based tic-tac-toe marking, and whiteboard drawing from raster images while maintaining safe execution flow and observable system state. Completion evidence was defined as end-to-end operation of all three modes through a single dashboard, preview and safety support for drawing jobs, and improved moving/idle feedback through RTDE-assisted monitoring.

2.1 Objectives

The primary objective of this project was to design and implement a web-based control platform for the Universal Robots UR3e that could support multiple task modes from a single browser interface.

The system was intended to:

- move the robot to calibrated board positions
- support chess-style suction pick/place interaction
- support tic-tac-toe movement and pen-based marking

- support whiteboard drawing from uploaded PNG/JPG images
- provide dry-run, control locking, and live status feedback
- remain usable in URSim as the primary development and validation environment

2.2 Target Success Criteria

The target success criteria were:

- board targets could be translated into calibrated poses and executable URScript
- chess and tic-tac-toe actions could be executed through the web interface without manual backend interaction
- uploaded raster images could be previewed and executed as drawing plans
- the system could reject oversized drawing jobs before execution
- the UI could reflect robot moving/idle state more accurately than fixed timing alone
- all three modes could operate within one dashboard and one backend architecture

2.3 Included Scope

The implemented scope includes:

- a React and TypeScript frontend dashboard
- a Node.js and Express backend API
- board calibration and whiteboard calibration profiles
- URScript generation and TCP dispatch to URSim
- RTDE-based live motion monitoring with dashboard fallback
- OpenCV contour extraction, stroke normalization, and planning
- simulator-based testing in URSim

2.4 Excluded Scope

The project does not include:

- full physical robot validation
- automatic calibration
- camera-based closed-loop correction
- robust SVG or text drawing support
- exact physical runtime prediction for whiteboard ETA

3 Project Overview and Technical Approach

This project is a web-based control platform for the Universal Robots UR3e focused on structured user interaction rather than low-level robot control alone. It combines frontend interaction, backend routing, calibration-aware mapping, URScript generation, and live robot status feedback into one system. The platform was designed for educational and laboratory workflows in which a user can select a task from a browser interface and have it translated into calibrated robot behavior.

3.1 Technical Approach

The technical approach was defined around three intentional layers. First, a web-based control layer in the React frontend and Express backend exposes robot operations through a single dashboard. Second, a calibration-based mapping layer converts board squares, tic-tac-toe cells, and normalized whiteboard coordinates into robot poses using manually defined anchors and local workspace frames. Third, an image-to-trajectory layer converts uploaded raster images into contour strokes, simplified drawing plans, safety-checked motion sequences, and executable URScript. These layers were designed to work together so that multiple task modes could share transport, control locking, status reporting, and calibration concepts instead of being implemented as unrelated demonstrations.

The implemented system supports three operating modes: chess-style board movement and suction-based pick/place, tic-tac-toe movement and pen-based marking, and whiteboard drawing from uploaded raster images. These modes were selected as complementary evaluation cases for the same architecture: target-based movement, grid-based marking, and image-driven trajectory generation. Together, they provide a broader evaluation of the control platform than a single-purpose demo would allow.

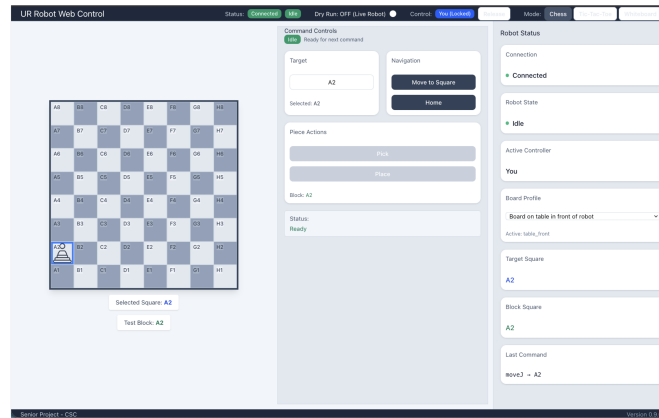
The main engineering result is a unified URSim-based platform that integrates multiple interaction modes within one interface and one backend architecture. Because validation was limited to URSim, the current result validates the software architecture, calibration workflow, and execution pipeline in simulation, but it does not establish final hardware performance.

Figure 1 shows the final dashboard with the three supported operating modes.

Multi-Mode Web Interface

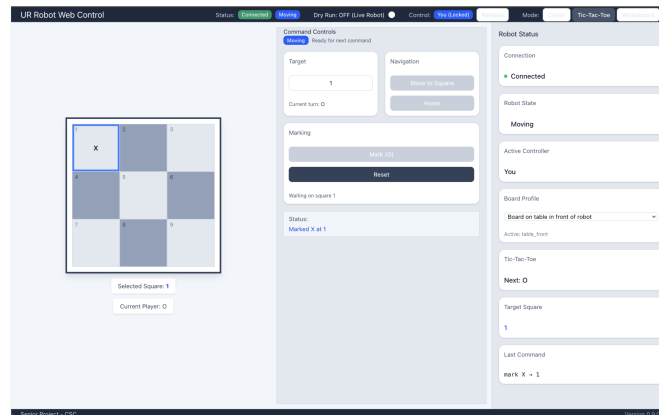
The same dashboard supports chess, tic-tac-toe, and whiteboard drawing modes.

(a) Chess Mode



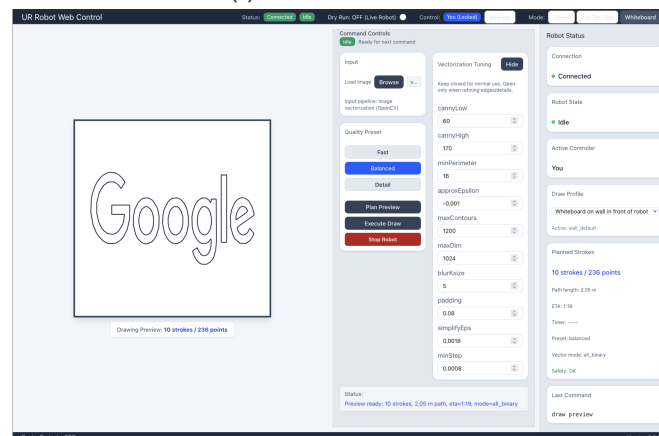
Move, pick, and place on a calibrated board.

(b) Tic-Tac-Toe Mode



Move to a cell and mark X or O trajectories.

(c) Whiteboard Mode



Upload, preview, and execute image-based drawing.

These screenshots show how one interface supports multiple robot interaction modes.

Figure 1: Multi-mode browser dashboard.

4 Background and Related Work

The project combines several common ideas from robotics, human-robot interaction, and software systems. At the interaction level, the system follows a supervisory-control pattern: the operator gives structured task commands through a software interface and monitors feedback from the robot instead of manually commanding every low-level motion. This is consistent with established human-robot interaction and telerobotics work, where usability, feedback, and the separation between operator intent and robot execution are central concerns [1, 2].

The calibration part of the project uses manually defined anchor points to construct a workspace frame. In this system, anchors define both the board frame for chess and tic-tac-toe interaction and the whiteboard plane for drawing. Once those frames are established, user-level targets can be converted into robot poses. This is simpler than full hand-eye calibration, but it is related to the broader calibration problem in robotics, where coordinate frames must be mapped reliably before robot motion can be trusted [4, 5].

The second major idea is task-to-command translation. Instead of issuing robot commands directly at the UI layer, the backend converts targets into URScript and sends them over TCP to URSim. The whiteboard mode adds an image-processing stage, where raster images are converted into contour strokes with OpenCV, normalized into the drawing frame, simplified, and then translated into robot motion. This is related to robotic sketching systems that combine image processing, path planning, and automatically generated robot control code [6]. In this project, the same architecture supports three task modes rather than a single-purpose drawing workflow.

This project builds directly on the official Universal Robots external interfaces, especially URScript for motion commands, RTDE for telemetry, and the dashboard server for fallback controller status.

5 Requirements and Evaluation Criteria

5.1 Functional Requirements

- Move the robot to selected board or drawing targets.
- Toggle suction for pick/place experiments.
- Draw X and O marks in tic-tac-toe mode.
- Accept raster image input and generate drawable strokes.
- Provide preview, execute, and stop actions for whiteboard drawing.
- Expose control and status through REST endpoints.

5.2 User Interaction Requirements

- Select operating mode and target from a single dashboard.
- Upload an image, tune parameters, preview, and execute.
- View connection state, robot state, command feedback, and ETA.
- Switch calibration profiles from the UI.

5.3 Non-Functional Requirements

- Safety: reject oversized drawing jobs before execution.
- Reliability: UI status should track motion as closely as possible.
- Performance: commands should dispatch quickly in simulator use.
- Maintainability: logic should stay modular across routes and robot interfaces.
- Portability: the system should work in URSim first, with a path toward hardware later.

5.4 Evaluation Criteria

- Accuracy: board targets should resolve to the intended region after calibration.
- Repeatability: repeated tasks should behave consistently in URSim.
- Status fidelity: moving/idle UI state should match observed robot behavior.
- Safety compliance: oversized jobs should be blocked with clear feedback.
- Usability: the main workflows should run without manual backend calls.

5.5 Requirements To Evaluation Traceability

Table 1 connects the main requirements to the design choices that address them, the modules that implement them, and the evidence collected during evaluation.

Table 1: Traceability from requirements to evaluation.

Requirement	Design Response	Implementation Modules	Evaluation Evidence and Limit
Board movement and marking	map UI targets through calibrated board frame	<code>move.js</code> , <code>squares.js</code>	URSim paths verified; hardware placement and pen contact not measured
Chess pick/place	share board calibration; separate pick/place actions	<code>move.js</code> , <code>urTcp.js</code> , board UI	UI flow and URScript sequencing verified; suction transfer remains hardware work
Whiteboard drawing	vectorize image, plan strokes, check safety, generate URScript	<code>draw.js</code> , <code>vectorizeImage.js</code> , <code>planner.js</code> , <code>whiteboard.js</code>	preview metrics measured for three images; physical drawing quality not measured
Safe execution	support preview, dry run, control lock, and job limits	<code>control.js</code> , <code>draw.js</code> , <code>App.tsx</code>	oversized-job checks and lock behavior exercised in URSim
Live status feedback	RTDE primary; dashboard fallback	<code>urRtde.js</code> , <code>urDashboard.js</code> , <code>state.js</code>	UI state followed simulator motion; exact latency not quantified
Dashboard usability	combine modes, target input, preview, execution, and status	<code>App.tsx</code> , <code>AppHeader.tsx</code> , <code>WorkspaceBoard.tsx</code>	all workflows exercised from browser; no formal user study

6 System Design

The system is organized as a layered web-to-robot architecture with five main component groups: the frontend dashboard, backend route handlers, calibration modules, the drawing pipeline, and robot interface modules. In repository terms, these responsibilities are primarily organized under `UI/src`, `backend/src/routes`, `backend/src/robot`, and `backend/src/draw`. The frontend collects user input and displays status, the route handlers validate requests and coordinate task execution, the calibration modules convert workspace coordinates into poses, the drawing pipeline converts raster inputs into normalized strokes, and the robot interface modules handle URScript transport and live state monitoring.

6.1 Architecture Style

The overall architecture is best described as a layered, pipeline-based architecture. The layered part separates user interaction, request handling, task planning, and robot communication. The React frontend is responsible for operator input and feedback, the Express route layer validates requests and coordinates workflows, the calibration and drawing modules transform user targets into robot-ready geometry, and the robot interface modules handle URScript, RTDE, and dashboard communication.

The pipeline part appears in how each task moves through the system. A board command follows a short pipeline from selected square or cell, to calibrated pose, to generated URScript, to URSim or UR3e execution. The whiteboard workflow extends that pipeline with image vectorization, stroke simplification, safety checks, and preview metrics before command generation. This architecture was chosen because it keeps each transformation explicit and replaceable. For example, a future hardware test can improve calibration or timing measurement without changing the frontend, and a future drawing workflow can replace the OpenCV vectorizer without rewriting the robot communication layer. Similar web-based robotic systems also emphasize modular architecture and browser-based accessibility for maintainability and portability [3].

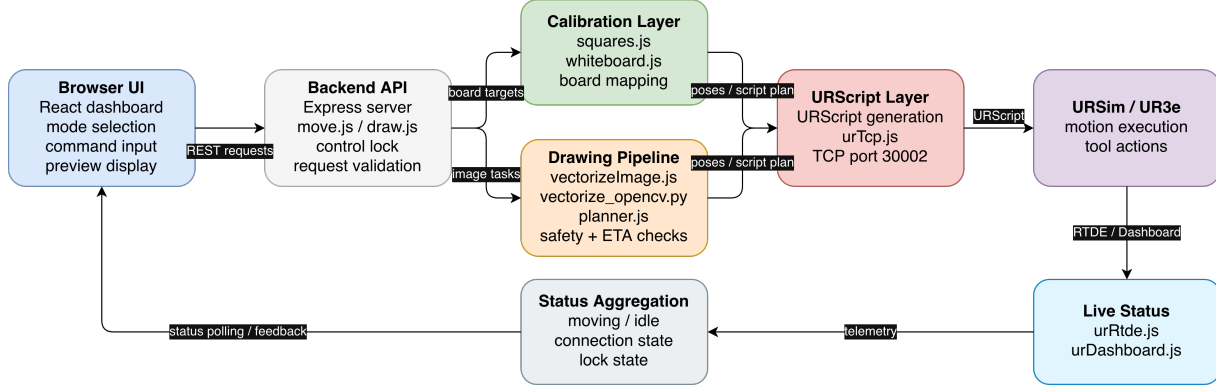
6.2 Software Design Blueprint

Figure 2 summarizes the implemented architecture as modules and data flows. The diagram shows the browser interface, backend API, calibration layer, drawing/vectorization pipeline, URScript transport, URSim/UR3e execution, and live status feedback path. Table 2 then expands that diagram into the specific frontend components, backend route controllers, calibration modules, drawing modules, and robot communication modules used in the implementation.

For chess and tic-tac-toe, the backend maps user-selected targets into calibrated poses and generates URScript directly. For whiteboard drawing, the backend first converts an uploaded raster image into contour strokes, simplifies them, checks safety limits, and only then generates robot motion. This design keeps image processing, geometry, and command transport separated instead of mixing them into a single control file.

UR3e Web Control: System Pipeline

From browser input to robot execution and live status feedback



Board tasks map directly through calibration. Whiteboard tasks add image vectorization and stroke planning. RTDE is the primary live status source.

Figure 2: Software blueprint and command/status flow.

6.3 Deployment View

Figure 3 complements the software pipeline by showing how the system is deployed at runtime. The browser UI communicates with the Node.js/Express backend through REST requests and status polling. The backend calls the Python/OpenCV vectorization pipeline as a local subprocess for image-based drawing, generates URScript for robot motion, and communicates with URSim or the UR3e controller through URScript TCP, RTDE, and dashboard interfaces.

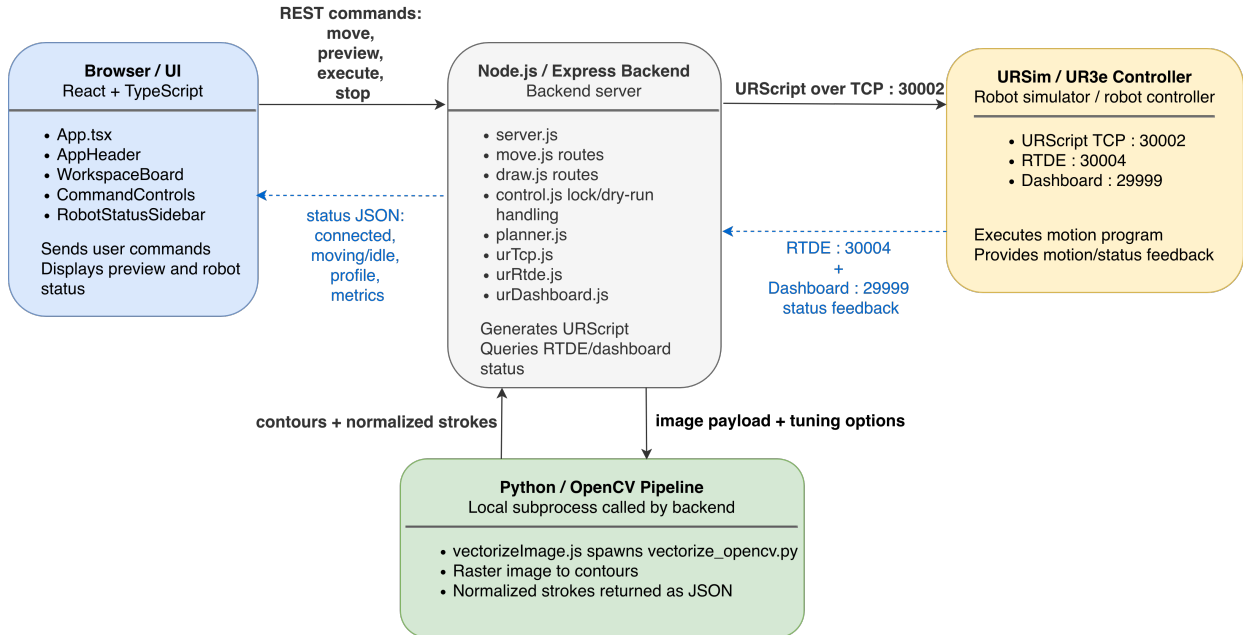


Figure 3: Runtime deployment view.

6.4 Module-Level Design

Figure 4 presents the same implementation from a software design perspective. Instead of showing runtime deployment, it shows the major source modules, their responsibilities, selected functions, and the dependencies between the frontend, route controllers, calibration modules, drawing pipeline, robot communication modules, and status feedback path.

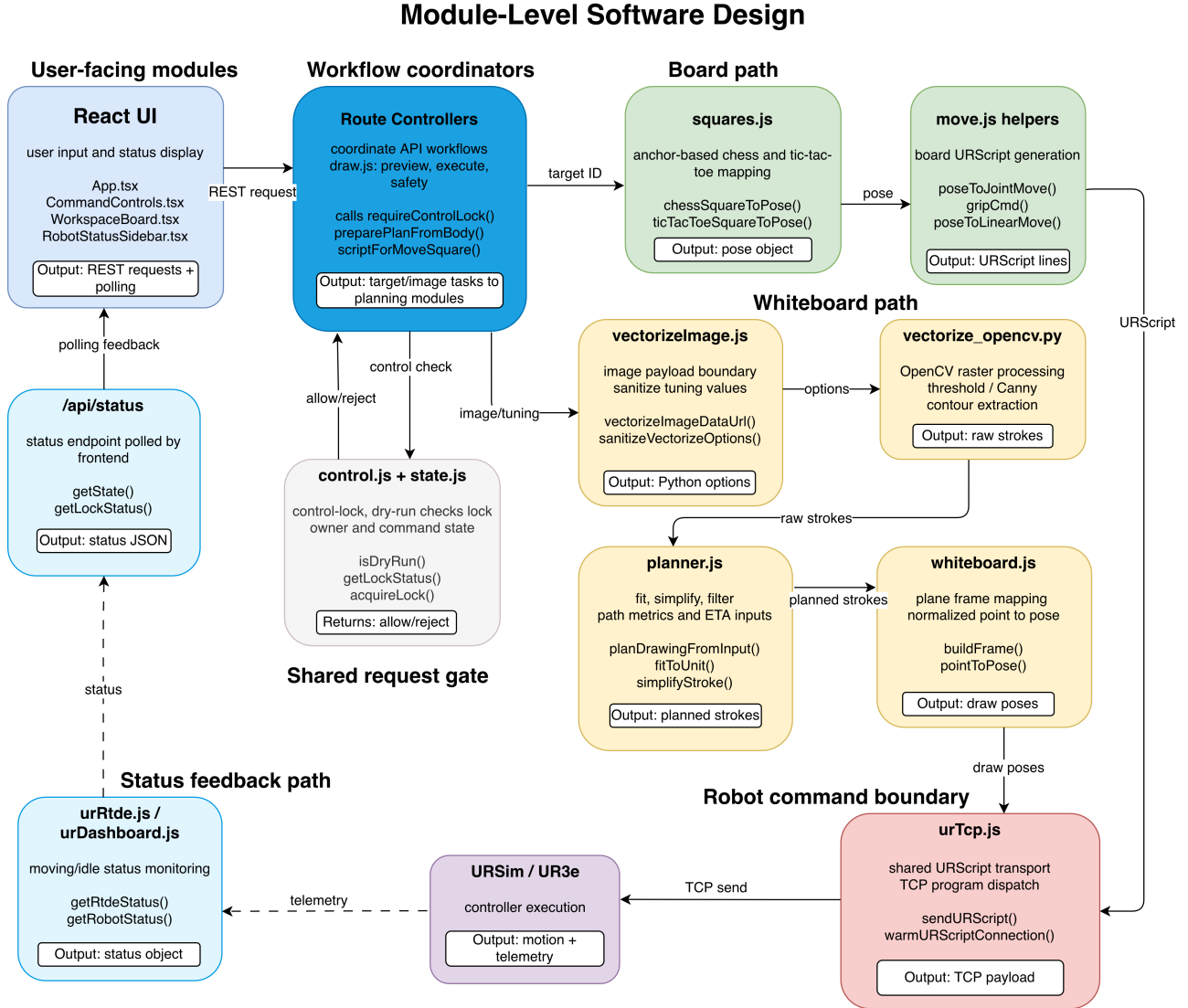


Figure 4: Module-level software design showing major modules, selected functions, outputs, and dependencies.

6.5 Module Responsibilities

Table 2: Software blueprint module responsibilities.

Component	File / Module	Responsibility	Interaction / Data Flow
Frontend shell	UI/src/App.tsx, AppHeader.tsx	mode selection, lock ownership, API coordination	sends REST requests and stores UI state
Workspace display	WorkspaceBoard.tsx	board/grid rendering and drawing preview	visualizes selected targets and strokes
Command controls	CommandControls.tsx	movement, upload, tuning, preview, execute, stop	builds request payloads
Status sidebar	RobotStatusSidebar.tsx	connection, moving/idle, lock owner, drawing metrics	displays backend status payloads
Movement API	backend/src/routes/move.js	chess and tic-tac-toe actions	maps square/cell requests to URScript
Draw API	backend/src/routes/draw.js	whiteboard preview, execute, stop, ETA, safety	coordinates vectorization, planning, and script generation
Control gate	backend/src/routes/control.js	dry-run and single-client ownership	approves or rejects requests
Board calibration	backend/src/robot/squares.js	board frame from anchors	maps target ids to board poses
Whiteboard calibration	backend/src/robot/whiteboard.js	whiteboard plane mapping	maps normalized strokes to draw poses
OpenCV vectorizer	backend/src/draw/vectorize_opencv.py	contour extraction	converts image input into contour strokes
Planner	backend/src/draw/planner.js	fit, simplify, measure strokes	returns strokes, points, path length, and ETA inputs
URScript transport	backend/src/robot/urTcp.js	command dispatch	sends URScript over TCP
RTDE monitor	backend/src/robot/urRtde.js	moving/idle inference	converts telemetry into UI status
Dashboard fallback	backend/src/robot/urDashboard.js	controller fallback status	supplies connection and running state

The code organization mirrors these component boundaries. Shared request and control-lock behavior is centralized in `backend/src/routes/control.js`, board-task logic remains in `move.js`, drawing-specific logic remains in `draw.js` and `backend/src/draw`, and transport / status logic remains isolated in `backend/src/robot`. This separation reduces coupling between task planning and robot communication while keeping shared control behavior in one place.

6.6 Data and Command Flow

The command path from a user action to URScript execution is:

1. The user selects a square, cell, or image task in the UI.
2. The frontend sends a REST request with the control token and current dry-run state.
3. The backend validates control ownership and request parameters.
4. Board tasks are mapped directly through `squares.js`; drawing tasks are vectorized, normalized, simplified, and safety-checked before pose generation.
5. URScript is generated by the route layer and sent through `urTcp.js` to URSim when dry-run is off.

The status feedback path returns information in the opposite direction:

1. URSim or the UR controller publishes motion and connection information through RTDE and dashboard interfaces.
2. `urRtde.js` infers moving/idle state from TCP and joint speeds, while `urDashboard.js` provides fallback controller status.
3. The backend status state is exposed through `GET/api/status`.
4. The frontend polls status and updates the header badges, command feedback, and robot status sidebar.

6.7 System Behavior During a Complete Workflow

A typical whiteboard drawing workflow begins with the user selecting Whiteboard mode in the browser interface, choosing the active whiteboard calibration profile, and uploading a raster image. The user then selects a preset or tuning values and presses *Plan Preview*. The frontend sends the image and tuning payload to the backend preview route. The backend validates the request, calls the Python/OpenCV vectorization step, converts the returned contours into normalized strokes, fits those strokes into the calibrated whiteboard frame, and computes preview metrics such as stroke count, point count, estimated path length, ETA, and safety status.

The intended workflow is for the user to review the preview before sending robot motion. If the preview is acceptable, the user presses *Execute Draw*. The backend repeats the same planning path used for preview, checks the control lock and safety limits, converts the planned strokes into URScript motion commands, and sends the program to URSim through the URScript TCP interface when dry-run is off. While the program runs, the backend monitors RTDE and dashboard status, and the frontend polls `/api/status` so the header and status sidebar update the connection state, moving/idle state, active profile, and command feedback. The board-based workflows follow the same overall behavior but skip the image-processing stage. For chess or tic-tac-toe, the user selects a square or cell in the UI, the backend maps that target through the active board calibration profile, generates the corresponding URScript movement or marking command, sends it to URSim when execution is enabled, and returns status information to the UI through the same feedback path.

Figure 5 summarizes the whiteboard path from image upload to planned robot motion.

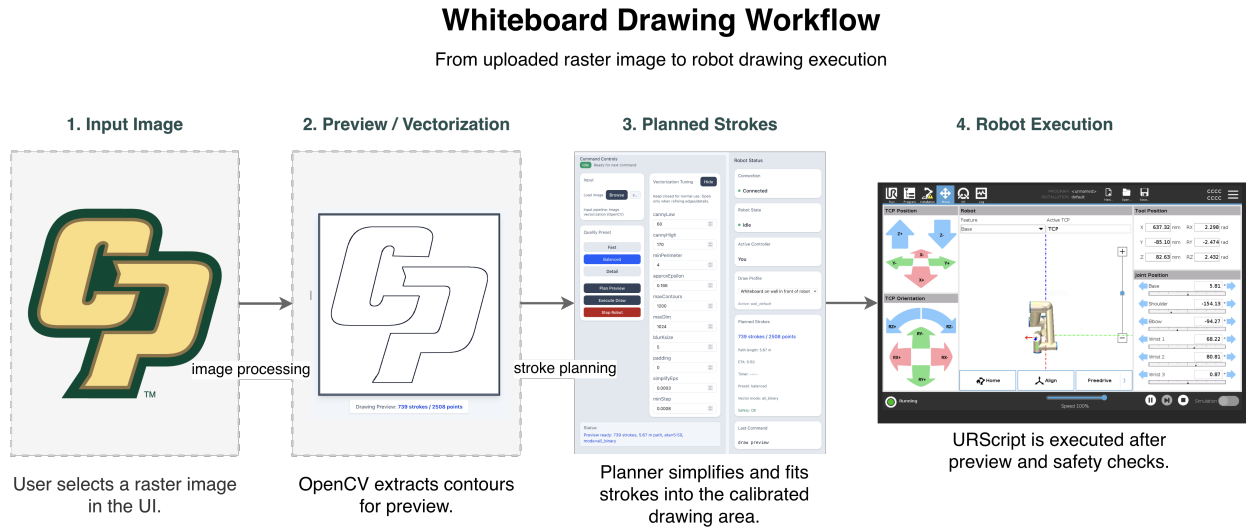


Figure 5: Whiteboard planning workflow.

6.8 Design Decisions and Trade-Offs

The design decisions below were made to keep the project feasible within the available lab access and to keep the software modular enough to extend later.

Table 3: Key design decisions and trade-offs.

Decision	Justification	Trade-Off
URSim-based development	develop communication, UI workflow, calibration math, scripts, and drawing pipeline without constant robot access	does not measure suction, pen contact, placement error, or real timing
Separate board and drawing pipelines	board tasks are target-to-pose; drawing tasks need vectorization, planning, and safety checks	some route coordination is repeated between <code>move.js</code> and <code>draw.js</code>
Manual anchor calibration	simple and transparent for a student project without camera sensing	accuracy depends on anchor setup; drift is not corrected automatically
Drawing preview workflow	exposes stroke count, point count, path length, ETA, and safety before motion	adds one planning step; preview quality still depends on image content
RTDE with dashboard fallback	RTDE gives better moving/idle feedback than fixed timers; dashboard gives fallback status	still inferred from telemetry thresholds, not ground-truth timing

6.9 Technology Selection Rationale

The implementation uses React with TypeScript, Node.js with Express, and Python/OpenCV because those technologies map naturally to the three main responsibilities in the system: an interactive browser dashboard, a web API that coordinates robot commands, and an image-processing pipeline for whiteboard drawing.

React and TypeScript were selected for the frontend because the interface is stateful rather than static. The UI must switch between chess, tic-tac-toe, and whiteboard modes, track the selected target, manage dry-run and control-lock state, display moving/idle feedback, and show drawing preview metrics. A component-based React interface made those states easier to organize than a server-rendered page or plain HTML form, while TypeScript helped keep command payloads and UI state more explicit. Node.js and Express were selected for the backend because the frontend communicates through JSON REST requests, and the backend’s main responsibilities are request validation, control locking, route coordination, URScript generation, TCP dispatch, and status polling. Keeping the browser UI and API layer in the JavaScript/TypeScript ecosystem reduced integration overhead between the frontend and backend.

Python/OpenCV was used only for the raster-to-stroke part of the whiteboard pipeline. This was preferable to implementing image processing directly in JavaScript because OpenCV provides mature operations for thresholding, contour extraction, and polygon approximation. The trade-off is that the system uses two runtimes and a subprocess boundary between Node.js and Python. That boundary was acceptable because image vectorization is a batch planning step rather than a continuous robot-control loop, and the modules exchange structured JSON before the backend continues with planning, safety checks, and URScript generation.

6.10 Software Engineering Rationale

The module boundaries were chosen to keep planning logic separate from communication side effects. Route handlers validate API requests and coordinate workflows, calibration and drawing modules convert user inputs into poses or strokes, and robot modules handle URScript TCP, RTDE, and dashboard communication. This separation makes the system easier to maintain because a board mapping, drawing planner, or status monitor can be adjusted without rewriting unrelated parts of the application. The frontend/backend split serves the same purpose: the React UI focuses on interaction and feedback, while the backend owns control locking, safety checks, command generation, and robot communication. Isolating chess, tic-tac-toe, drawing, calibration, and transport code also creates clearer extension points for future board profiles, SVG/text input, hardware timing measurements, or alternate robot interfaces.

6.11 Software Engineering Practices and Metrics

The repository is organized around the same boundaries shown in the module-level design. The frontend is contained in `UI/src`, with `App.tsx` coordinating application state and smaller components handling the header, command controls, workspace board, and status sidebar. Backend route controllers are in `backend/src/routes`; robot communication, calibration, and shared state are in `backend/src/robot`; and the whiteboard vectorization/planning code is in `backend/src/draw`. This layout keeps UI rendering, API coordination, task planning, and robot communication separately modifiable.

Testing used a combination of build/syntax checks and manual end-to-end validation. The frontend build checks TypeScript and Vite packaging, while the backend check verifies that the JavaScript source parses correctly. The main project validation then exercised representative workflows manually through the browser and URSim because the most important behavior involved integrated UI input, REST routing, URScript generation, simulator motion, and status feedback. The final documented validation set included at least twelve manual workflow checks across chess, tic-tac-toe, whiteboard preview/execution, control-lock behavior, and moving/idle status observation.

Table 4: Software implementation metrics.

Metric	Value Used in Report
Frontend source modules	14 TypeScript/TSX files under <code>UI/src</code>
Backend source modules	13 JavaScript/Python files under <code>backend/src</code>
Approximate source size	4,905 lines total: 2,176 frontend and 2,729 backend
Application API endpoints	17 robot/drawing endpoints, plus a health endpoint
Validation experiments	at least 12 manual workflow checks
Whiteboard images tested	at least 10 logo/image inputs explored; 4 examples shown in the report, with 3 bundled in the repository

These metrics are approximate implementation metrics rather than a formal productivity measure. Their purpose is to make the project scale clearer and to show that the final system was not a single script: it consisted of separate UI components, route controllers, calibration modules, planning modules, and communication modules that could be tested and changed independently.

7 Implementation

7.1 Backend API Structure

The backend entry point is `backend/src/server.js`. It mounts `move.js` at `/api` and `draw.js` at `/api/draw`, starts the RTDE monitor, and optionally warms the URScript socket before the first command is sent. Shared request gating is implemented in `backend/src/routes/control.js`. In that module, `isDryRun(req)` maps the query parameter `dryRun=1` into a non-executing path, `getControlToken(req)` reads `x-control-token` or a query token, and `requireControlLock(req, res)` either refreshes the active lock or rejects the request with HTTP 423 if another client owns control. This keeps movement and drawing routes consistent without duplicating lock logic.

Listing 1: Backend control-lock check before command execution.

```
function requireControlLock(req, res) {
  const token = getControlToken(req);
  if (!token) {
    res.status(401).json({ ok: false, error: "Control token required" });
    return null;
  }
}
```

```

const lock = getLockStatus(token);
if (!lock.held && !acquireLock(token).ok) {
  res.status(423).json({
    ok: false,
    error: "Control locked by another client",
  });
  return null;
}
if (lock.held && !lock.yours) {
  res.status(423).json({
    ok: false,
    error: "Control locked by another client",
  });
  return null;
}
refreshLock(token);
return token;
}

```

Listing 1 shows one backend implementation detail used before command execution. Movement and drawing routes call this helper before sending URScript, so a command is allowed only when the request includes the active control token. If another browser client owns the lock, the backend returns HTTP 423 and does not continue to planning or robot dispatch. This design keeps concurrency handling in one module instead of scattering duplicate lock checks across every route.

The movement API in `backend/src/routes/move.js` supports routes such as:

- `POST/api/moveSquare/:id`
- `POST/api/pick/:id`
- `POST/api/place/:id`
- `POST/api/ttt/move/:id`
- `POST/api/ttt/mark/:id`
- `GET/api/status`
- lock and board-profile routes

The drawing API in `backend/src/routes/draw.js` supports:

- `POST/api/draw/preview`
- `POST/api/draw/execute`
- `POST/api/draw/stop`

- whiteboard profile routes

7.2 Calibration and Mapping

Board calibration is implemented in `backend/src/robot/squares.js`. The board frame is built from three anchors, `A1`, `B1`, and `A2`, using vector normalization and orthogonalization rather than assuming the anchors already form a perfect rectangle. `buildBoard(...)` computes `fileVec`, `rankVec`, and a board normal from those anchors, and it also estimates small per-square orientation drift through `orientationAt(...)` so that rotation can vary across the board when `orientationInterpolation` is enabled.

Chess square conversion uses the calibrated origin plus scaled file and rank basis vectors: conceptually, `pose = A1 + file * fileVec + rank * rankVec`, followed by a small orientation adjustment from `orientationAt(...)`. This avoids storing 64 fixed poses and makes the board mapping depend on a small number of anchors instead of many hand-entered coordinates. The same idea is reused for tic-tac-toe through `ticTacToeSquareToPose(...)`, which maps cells 1–9 onto a 3×3 sub-grid inside the calibrated board frame.

```
pos = A1 + file * fileVec + rank * rankVec
rot = orientationAt(file, rank)
```

Whiteboard calibration is implemented in `backend/src/robot/whiteboard.js`. That module computes `uVec`, `vVec`, and a plane normal from `topLeft`, `topRight`, and `bottomLeft`. The drawing planner works in normalized (`u`, `v`) coordinates, and `pointToPose(...)` clamps those coordinates into the calibrated frame before converting them to robot poses. This design keeps image processing independent from the physical board dimensions: the OpenCV pipeline can produce normalized strokes, while the calibration module alone decides where those strokes appear in robot space.

Figure 6 shows the anchor-point layout used to define the local board and whiteboard frames.

Calibration and Anchor Point Mapping

Anchor points define the workspace frame used to convert user-level targets into robot poses.

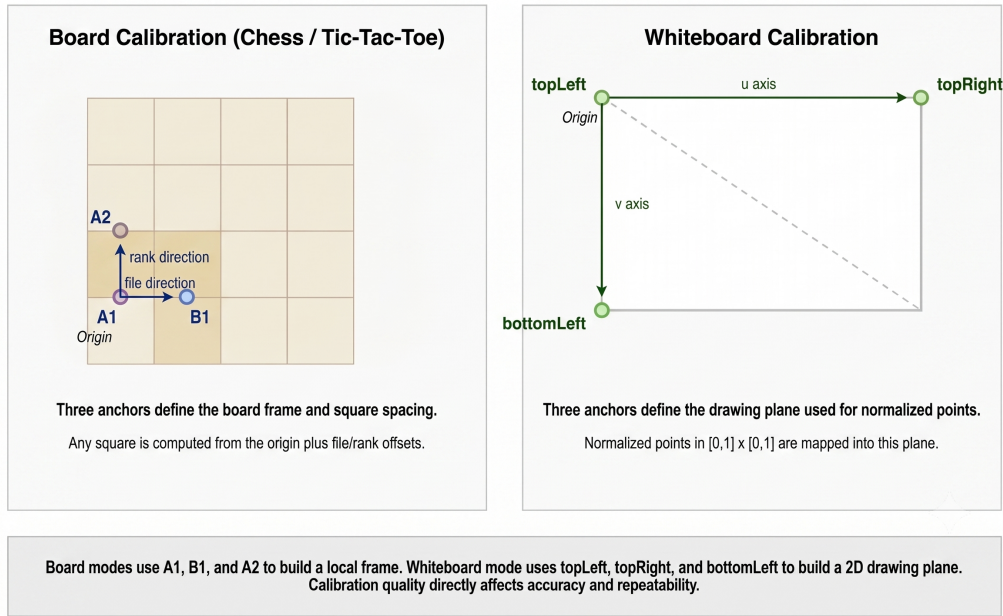


Figure 6: Anchor-based workspace calibration.

7.3 Whiteboard Drawing Pipeline

The whiteboard path is implemented as a staged pipeline across `vectorizeImage.js`, `vectorize_opencv.py`, `planner.js`, and `draw.js`. `vectorizeImage.js` first validates that the frontend provided a `base64 data:image/...` payload, then sanitizes UI tuning values before they reach Python. For example, `sanitizeVectorizeOptions(...)` clamps values such as `maxDim`, `blurKsize`, `cannyLow`, `cannyHigh`, and `approxEpsilonFrac` into safe ranges so that the OpenCV stage cannot be called with obviously bad parameters.

After contour extraction, `planDrawingFromInput(...)` in `backend/src/draw/planner.js` normalizes and filters the stroke set. The planner first fits the incoming strokes into a padded unit square, then simplifies each stroke with Ramer-Douglas-Peucker reduction, then removes leftover tiny segments using `minStep`. This processing is intentionally separated from OpenCV because contour extraction and robot path preparation are different concerns: OpenCV finds candidate outlines, while the planner turns those outlines into a bounded path that can be measured, previewed, and later converted into robot motion. The default values in the current planner are `padding = 0.08`, `simplifyEpsilon = 0.0018`, and `minStep = 0.0008`.

Listing 2: Short whiteboard planner excerpt from `planner.js`.

```
function planDrawingFromInput(input = {}) {
  let strokes = normalizeIncomingStrokes(input.strokes);
  strokes = fitToUnit(strokes, padding)
    .map((stroke) => simplifyStroke(stroke, epsilon, minStep))
    .filter((stroke) => stroke.length > 1);

  const pointCount =
    strokes.reduce((total, stroke) => total + stroke.length, 0);
  return { strokeCount: strokes.length, pointCount, strokesNormalized: strokes };
}
```

The route-level planner entry in `backend/src/routes/draw.js` combines vectorization and planning in one shared preparation path for both preview and execute. This was an important implementation decision because it prevents the preview from being a separate approximation of the executed path. The same image payload and tuning values are used before safety checks, ETA estimation, and URScript generation, so differences between preview and execution are limited to dispatch behavior rather than planning logic.

Safety checks in `draw.js` then measure stroke count, point count, physical path length, and script-line count before execution. The same route also computes preview ETA using draw speed, travel speed, stroke-lift overhead, and a per-point overhead term rather than using a fixed constant delay.

The whiteboard UI also exposes tuning controls that are important for difficult images. Three preset bundles, `fast`, `balanced`, and `detail`, are defined in `UI/src/config/draw.ts`, and the advanced panel in `UI/src/components/CommandControls.tsx` exposes the underlying parameters directly when manual refinement is needed. The vectorization controls, `maxDim`, `blurKsize`, `minPerimeterPx`, `approxEpsilonFrac`, and `maxContours`, affect how OpenCV extracts and approximates contours from the raster image. The UI also shows `cannyLow` and `cannyHigh`, which matter when the pipeline uses Canny-based contour extraction, although the current validation results used `outlineBinary=true` and `externalOnly=false`, so blur, contour approximation, and contour filtering were the dominant vectorization settings. The planner controls, `padding`, `simplifyEpsilon`, and `minStep`, act after contour extraction by fitting the strokes into the whiteboard frame and removing redundant or overly short segments. This distinction mattered in validation because `sample3.png` required both vectorization and planner tuning to produce a clean four-stroke preview.

Table 5 summarizes the advanced whiteboard tuning parameters. These controls are not the main user workflow, so the report does not treat them as a separate algorithmic contribution. However, they are important implementation details because they explain how the system

transitions from a general-purpose preset workflow to manual refinement when a difficult image produces an unusable preview.

Table 5: Advanced whiteboard tuning parameters.

Parameter	Stage	Effect on Output	Implemented In
<code>maxDim</code>	preprocessing	caps image size before contour extraction	<code>draw.ts</code> , <code>vectorizeImage.js</code> , <code>vectorize_opencv.py</code>
<code>blurKsize</code>	preprocessing	blurs noise before contour extraction	<code>vectorize_opencv.py</code>
<code>cannyLow</code> , <code>cannyHigh</code>	alternate edge path	set Canny edge thresholds	<code>vectorize_opencv.py</code>
<code>minPerimeterPx</code>	contour filtering	removes tiny noisy contours	<code>vectorize_opencv.py</code>
<code>approxEpsilonFrac</code>	contour approximation	lower keeps detail; higher simplifies more	<code>vectorize_opencv.py</code>
<code>maxContours</code>	contour limiting	caps dense jobs by keeping largest contours	<code>vectorize_opencv.py</code>
<code>padding</code>	planner fitting	keeps drawing away from frame edges	<code>planner.js</code>
<code>simplifyEpsilon</code>	planner simplification	removes intermediate path points	<code>planner.js</code>
<code>minStep</code>	planner cleanup	removes very short leftover segments	<code>planner.js</code>

From an implementation perspective, these controls follow one consistent path through the system. The frontend stores them in the `DrawTuning` type, loads preset values from `UI/src/config/draw.ts`, and sends them in the preview / execute payload from `UI/src/App.tsx`. The backend then sanitizes the vectorization settings in `vectorizeImage.js`, applies the OpenCV-specific controls in `vectorize_opencv.py`, and applies the planner-specific controls in `planner.js`. This is why preview and execute remain consistent: both operations use the same tuning payload and the same preparation path in `backend/src/routes/draw.js`.

7.4 URScript Generation

Movement routes generate URScript directly from calibrated poses. The `poseToJointMove(...)` helper emits joint moves for hover and travel, while `poseToLinearMove(...)` emits linear moves for short contact motions. Tic-tac-toe marking is built in the movement route: an X is represented as two diagonal contact strokes, and an O is represented as a segmented circular path. These shapes are simple by design because the purpose of the mode is to validate bounded marking behavior, not to implement a general drawing engine.

For whiteboard drawing, `buildScriptFromStrokes(...)` in `backend/src/routes/draw.js` converts each normalized stroke into an approach-contact-trace-lift sequence. The robot first travels above the board, then lowers to the drawing surface, traces the stroke with linear moves, and lifts away before the next stroke. This separation between travel motion and contact motion is important because it allows `evaluateSafety(...)` and `estimateDrawTiming(...)` to reason about stroke count, path length, and script-line count before execution. It also makes the generated program easier to inspect because each stroke has a predictable structure.

```
movej(firstUp) -> movel(firstDown) -> trace stroke -> lift
```

7.5 Robot Communication and Status

URScript transport is implemented in `backend/src/robot/urTcp.js`. Live motion state is monitored through `backend/src/robot/urRtde.js`, which connects to port 30004, negotiates RTDE, subscribes to a small set of output fields, and infers moving versus idle state from TCP speed and joint speed thresholds. Fallback controller status is provided by `backend/src/robot/urDashboard.js`.

RTDE was introduced because dashboard polling and fixed timing alone were not accurate enough for consistent moving / idle feedback in the UI. The combination of RTDE as the primary source and dashboard as fallback improved state reporting without changing the motion generation path itself.

Listing 3: RTDE moving/idle status inference excerpt.

```
const speedMag = magnitude(values.actual_TCP_speed || []);
const jointSpeedMax = maxAbs(values.actual_qd || []);
const now = Date.now();

const aboveStart =
  speedMag >= this.speedStartEps ||
  jointSpeedMax >= this.jointSpeedStartEps;
const belowStop =
  speedMag <= this.speedStopEps &&
  jointSpeedMax <= this.jointSpeedStopEps;

if (aboveStart) this.lastMotionAt = now;

let moving = this.state.moving;
if (aboveStart) {
  moving = true;
} else if (belowStop) {
```

```

const recentMotion =
  this.lastMotionAt && now - this.lastMotionAt <= this.movingHoldMs;
moving = !!recentMotion;
}
this.state.moving = !!moving;

```

Listing 3 shows the status-monitoring logic used by the RTDE client. The backend does not mark the robot as moving only because a command was sent; it also checks TCP and joint-speed telemetry from the controller. This is why the moving/idle badge can return to *Idle* when simulator motion stops, while the whiteboard ETA remains a planner estimate based on generated path length and not a live timing measurement.

During URSim tests, the observed status feedback was responsive enough for operator-facing use: after a command was sent, the UI changed from *Idle* to *Moving* during simulator motion and returned to *Idle* after the motion stopped. The final documented validation set included at least twelve manual workflow checks across chess move/pick/place actions, tic-tac-toe move/mark actions, and whiteboard preview or tuning runs. In the motion runs where RTDE/dashboard feedback was visible, the moving/idle transition was observed consistently, usually returning to *Idle* as soon as simulator motion stopped. This responsiveness was assessed qualitatively from manual runs and screenshots, not from timestamped latency logs. Therefore, the report does not claim a measured RTDE latency. A physical follow-up should log command dispatch time, RTDE sample time, and browser update time so that status latency and false moving/idle transitions can be quantified.

7.6 Frontend and User Interface

The frontend is implemented in React and TypeScript. The main dashboard logic is in `UI/src/App.tsx`, while supporting components separate the header, board rendering, command controls, and robot status display. Figma was used to plan the layout and grouping of controls before the final interface was implemented. The final UI emphasizes mode switching, target selection, dry-run toggling, calibration profile selection, status display, and the drawing preview workflow.

8 Experiments and Validation

8.1 Experiment Structure

Validation combined integrated browser-to-URSim workflow checks for all three operating modes with measured whiteboard planner outputs from the application preview. Each workflow was exercised through the normal browser interface rather than isolated backend

calls, so the evidence reflects the full system path. Because physical robot testing was outside the evaluation scope, this section separates simulator evidence from hardware metrics that remain unmeasured.

Table 6: Experiment structure.

Workflow	What Was Validated	Measured or Observed Outputs	What Was Not Validated
Chess move / pick / place	UI route, URScript, calibrated URSim path	selected square, generated script, simulator response	pickup success, placement error, cycle time
Tic-tac-toe marking	cell selection, mark script, bounded path	selected cell, generated mark script, simulator response	mark shape, pen pressure, contact consistency
Whiteboard drawing	raster-to-stroke planning, safety checks, script generation	strokes, points, script lines, path length, ETA, preview image	resemblance, contour error, real ETA error
Status feedback	RTDE/dashboard path back to UI	moving/idle state during motion	latency or controller timing error

Table 7 summarizes the validation status by requirement area. The status is separated from the evidence because some requirements were fully exercised in URSim, while hardware-dependent outcomes could only be identified as future measurements.

Table 7: Requirement validation status summary.

Requirement Area	Validation Performed	Status
Browser control interface	exercised chess, tic-tac-toe, and whiteboard workflows through the React UI and REST API	Fully validated in software
Board target movement	selected chess squares and tic-tac-toe cells generated calibrated URScript poses and URSim motion	Fully validated in URSim
Chess pick/place	generated move, pick, suction, and place commands through the same route and calibration path	Partially validated; suction transfer not tested
Tic-tac-toe marking	generated bounded marking trajectories for selected cells and observed simulator motion	Partially validated; physical mark quality not tested
Whiteboard drawing	generated previews, stroke metrics, path length, ETA, safety checks, and drawing scripts from sample images	Partially validated; physical drawing output not tested
Status feedback	observed moving/idle feedback returning from RTDE/dashboard monitoring into the UI	Partially validated; latency not measured
Physical hardware metrics	identified required measurements for placement, repeatability, suction, and drawing quality	Not validated in this project

Table 8 lists representative test scenarios in a pass/fail format. These are not separate unit tests; they are manual integrated workflow checks performed through the browser UI and URSim.

Table 8: Representative validation test matrix.

Scenario	Expected Result	Evidence	Result
Move to square	selected board target maps to a calibrated pose and sends a URScript move	chess validation figure and dry-run script	Pass in URSim
Pick/place sequence	pick and place actions generate motion plus suction on/off commands	chess pick/place validation figure	Pass in software/URSim; suction not hardware-tested
Tic-tac-toe mark	selected cell generates bounded X/O marking trajectory	tic-tac-toe validation figure and script excerpt	Pass in URSim
Whiteboard preview	uploaded image produces strokes, points, path length, ETA, preview, and safety status	whiteboard result figures and metrics table	Pass
Whiteboard execution	drawing command dispatches to URSim and UI returns to idle after motion	Figure 9	Pass in URSim
Over-limit drawing rejection	drawing jobs beyond stroke, point, path-length, or script-line limits are rejected before robot dispatch	draw route safety checks	Pass by implementation check
Control-lock conflict	requests without the active control token are rejected	Listing 1	Pass by backend behavior
Moving/idle status update	RTDE/dashboard feedback updates the UI during and after motion	manual runs and status screenshots	Pass qualitatively; latency not measured

8.2 Why URSim Validation Was Still Meaningful

URSim validation was meaningful because the project goal was not only to move a physical arm; it was also to verify the software path that turns a browser action into a robot command. The simulator allowed the full application workflow to be exercised: frontend selection, REST request handling, control-lock checks, calibration mapping, URScript generation, command dispatch, RTDE/dashboard status monitoring, and UI feedback. Since URSim uses the

same URScript programming model and the same style of controller interfaces as a Universal Robots controller, these tests provided useful evidence that the command pipeline and status pipeline were connected correctly before any physical deployment.

Physical testing was not completed within the project timeline because live UR3e testing requires supervised lab access, recalibration of the physical setup, tool mounting checks, and safety review before generated motion can be run. For that reason, hardware-specific outcomes are marked as unvalidated rather than partially validated. If supervised access is available, the same test matrix in Table 6 can be repeated on the real robot with added measurements for placement error, suction success, pen contact quality, emergency-stop behavior, and real execution time.

8.3 Calibration Accuracy Discussion

The calibration evidence in this report supports region-level accuracy in URSim rather than measured millimeter-level accuracy. For board tasks, the expected behavior of the anchor-based approach is that targets land near the intended square or cell center when the A1, B1, and A2 anchors are placed accurately and the board remains fixed. Because all square and cell poses are interpolated from the same local basis vectors, small anchor errors propagate across the board: an error in the origin shifts the whole grid, while errors in the horizontal or vertical anchors can skew scale, rotation, or orthogonality.

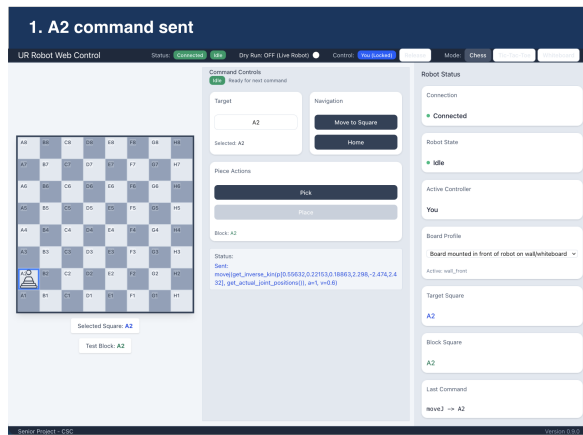
On physical hardware, additional error would come from TCP/tool offset, suction or pen mounting, board/whiteboard alignment, surface drift after calibration, and contact effects during pickup or drawing. For that reason, this project does not claim measured physical placement accuracy. A reasonable future hardware target would be to keep board placement error within approximately 5–10 mm of the intended square or cell center, repeated-target variation within approximately 3–5 mm, and whiteboard marking or drawing deviation within approximately 10–15 mm for simple shapes. These values should be treated as target evaluation metrics rather than measured results. A future hardware validation should move repeatedly to known square centers, tic-tac-toe cell centers, and whiteboard fiducials, then report mean error, maximum error, repeatability, suction success rate, and marking accuracy in millimeters. That would convert the current qualitative simulator calibration check into a quantitative calibration metric.

8.4 Simulator Execution and Command Evidence

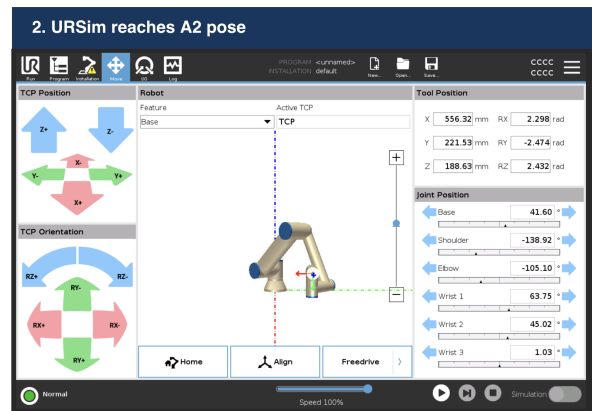
Figures 7 and 8 add simulator-side evidence for the board workflows. The screenshots were captured from the integrated browser interface and URSim, showing selected tasks, command feedback, simulator pose or motion state, and returned UI status.

Chess Pick/Place Validation Evidence

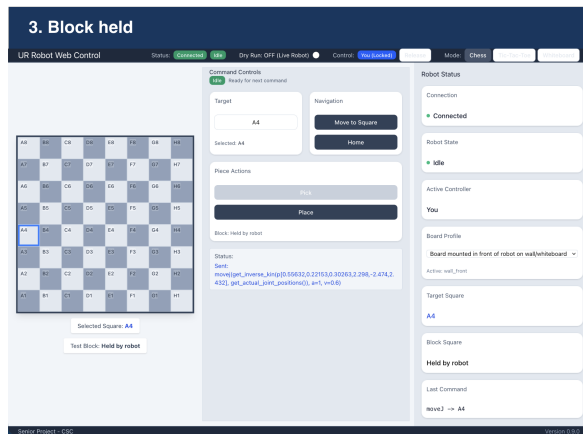
Selected square, URSim pose, held state, and final release are shown as one workflow.



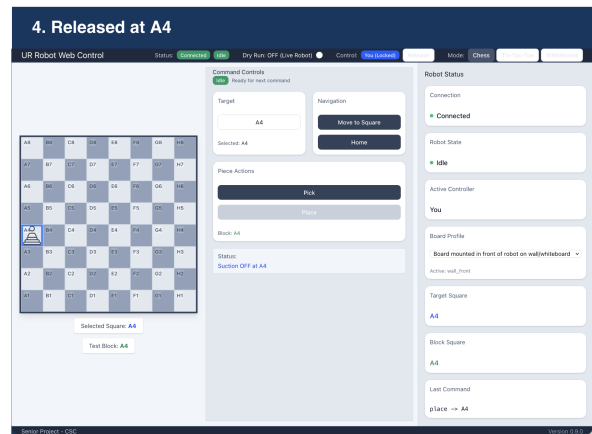
UI selects A2 and shows generated move command.



Simulator pose confirms target motion path.



Pick action updates held-block state.

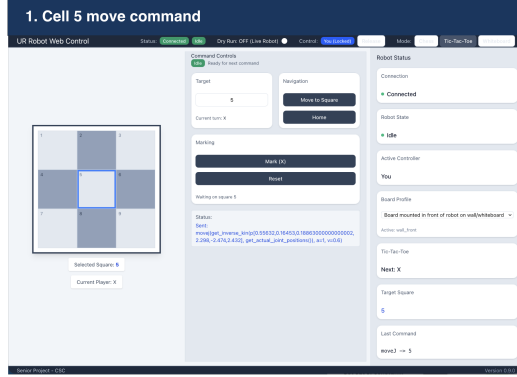


Place action updates the board state at A4.

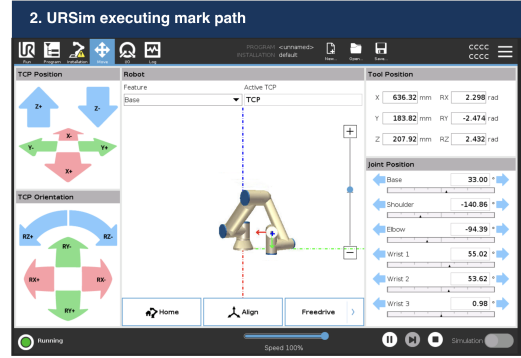
Figure 7: Chess pick/place validation evidence.

Tic-Tac-Toe Marking Validation Evidence

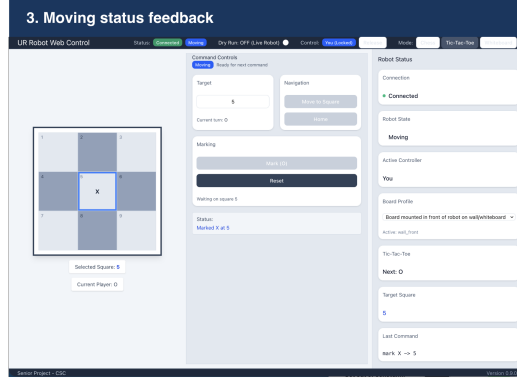
Cell selection, simulator execution, moving status, and completed mark are shown together.



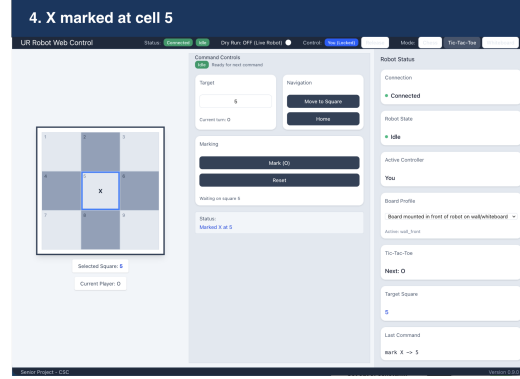
UI selects cell 5 and sends moveJ.



Simulator shows the active marking motion.



UI reflects moving state during execution.



Workflow returns to idle after mark completes.

Figure 8: Tic-tac-toe marking validation evidence.

The following URScript excerpts were captured from backend dry-run responses using the same `wall_front` board profile shown in the screenshots. Dry-run mode prevents TCP dispatch but uses the same route handlers, calibration mapping, and URScript generation path used when execution is enabled.

Listing 4: Short URScript excerpt for chess movement and suction actions.

```
# moveSquare A2: calibrated board pose
movej(get_inverse_kin(p[0.55632,0.22153,0.18863,...]), a=1, v=0.6)
# pick/place: suction output toggle
set_digital_out(0, True)
set_digital_out(0, False)
```

Listing 5: Short URScript excerpt for tic-tac-toe cell 5 marking.

```
# move to cell 5, then trace a contact stroke
movej(get_inverse_kin(p[0.55632,0.16453,0.18863,...]), a=1, v=0.6)
movel(p[0.63632,0.14515,0.20801,...], a=0.6, v=0.015)
```

```
...
move1(p[0.63632,0.14515,0.16925,...], a=0.6, v=0.015)
```

These excerpts are intentionally shortened. Their purpose is not to reproduce every generated line, but to show what the backend produced after the UI action passed through routing, calibration, and command generation. Listing 4 shows that a selected chess square becomes a calibrated `movej` pose and that pick/place actions become digital-output suction commands. Listing 5 shows that one tic-tac-toe cell selection expands into travel motion plus bounded linear contact strokes for marking.

The tic-tac-toe evidence shows the UI changing to *Moving* during execution and returning to *Idle* afterward. A quantified RTDE latency log was not collected, so status evidence is limited to observed UI transitions with dashboard fallback available in the backend.

8.5 Whiteboard Execution Evidence

Figure 9 adds execution evidence for the whiteboard workflow. The panels show the browser UI during a live drawing command, URSim running the generated trajectory, and the UI returning to *Idle* after simulator motion stops.

8.6 Validation Boundary Summary

Table 9: Validation boundary summary.

Area	Evidence in This Report	Boundary / Limitation
Board mapping	square/cell selections generated URScript paths and simulator responses	no millimeter-error measurement
Chess pick / place	same UI/backend flow as board movement	suction transfer not tested
Tic-tac-toe marking	X/O trajectories generated for selected cells	mark quality and pen contact not tested
Whiteboard preview/execute	strokes, points, path length, ETA, safety, and script size reported	physical drawing output not measured
RTDE moving / idle feedback	UI reflected moving/idle transitions with dashboard fallback	latency not quantified

Whiteboard Execution Validation Evidence

Web interface status and URSim motion are shown during one drawing execution.

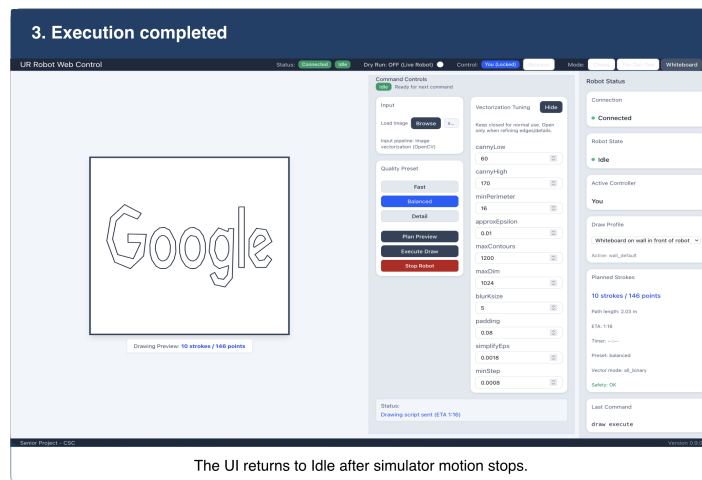
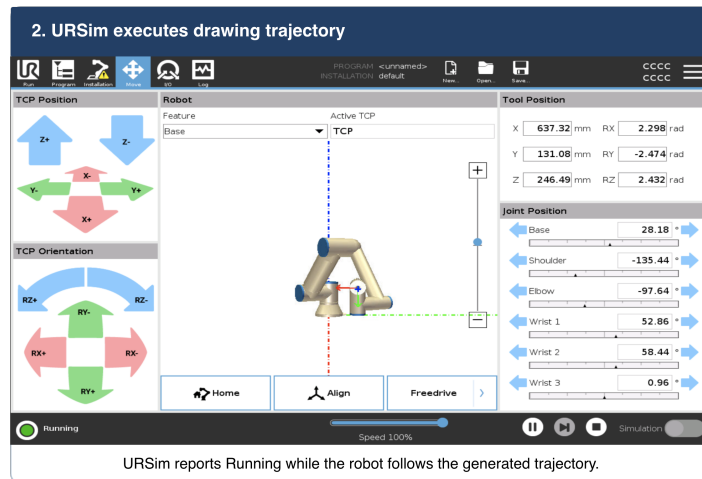
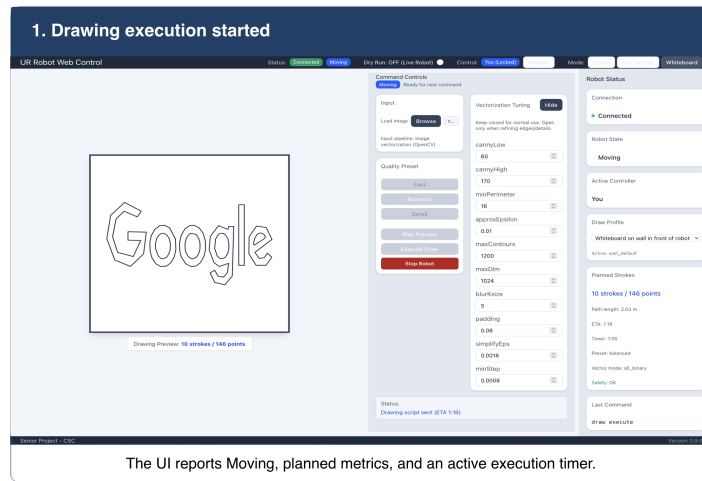


Figure 9: Whiteboard execution evidence showing UI motion status, URSim execution, and returned idle state.

8.7 Metric Coverage Summary

Table 10 separates collected measurements from metrics that require physical robot testing.

Table 10: Measured and unmeasured evaluation metrics.

Metric Area	Evidence Collected	Not Measured in This Project
Accuracy	board/cell targets reached intended simulator regions; previews showed stroke fit	placement error, pen offset, suction alignment, camera-verified error
Timing	whiteboard ETAs: 76 s, 176 s, and 156 s	real execution time, ETA error, network/controller latency
Repeatability	same inputs and profiles produced same URSim paths	trial variance, hardware drift, long-session repeatability
Drawing complexity	previews reported strokes, points, script lines, path length	line quality, missed strokes, marker behavior
Status feedback	moving/idle status observed in UI with dashboard fallback	status latency, false transitions, timing accuracy

8.8 Whiteboard Visual Results

Figure 10 shows the three validation images and the corresponding planned previews generated by the current pipeline. `sample1.jpg` and `sample2.png` produced usable previews with the default `balanced` preset. `sample3.png` did not: the logo required manual refinement because the default settings over-segmented the shape into many disconnected contour fragments.

Whiteboard validation examples

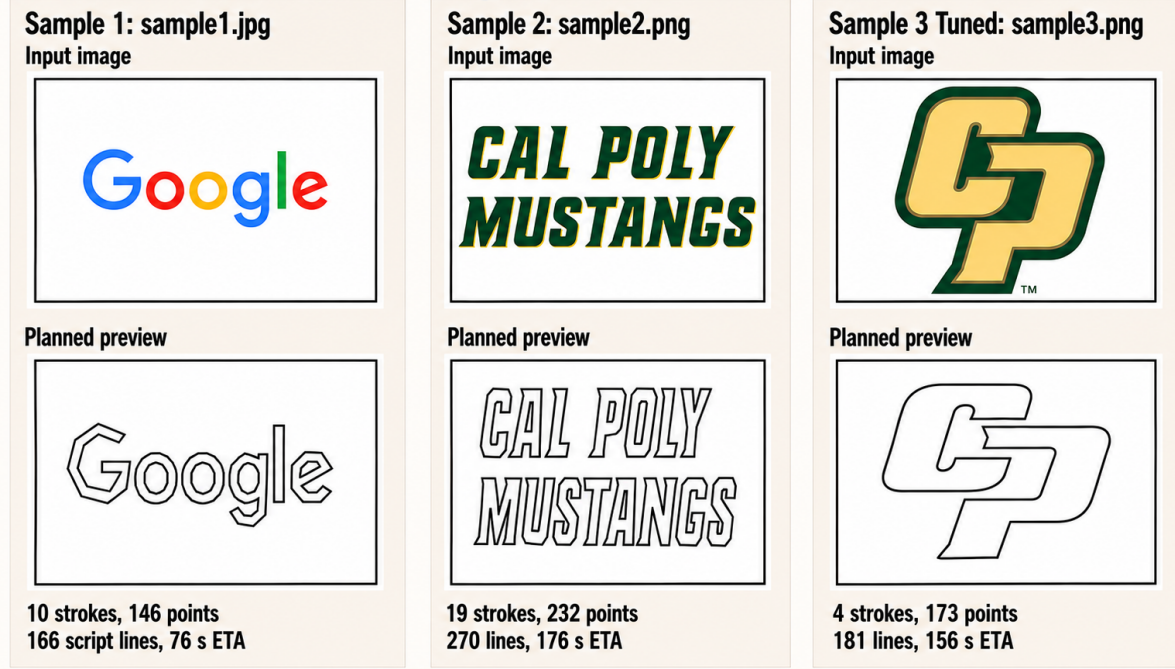


Figure 10: Input images and planned whiteboard previews.

8.9 Measured Whiteboard Planner Results

To provide concrete measured outputs, the bundled sample images were tested through the current whiteboard planner using the default `wall_default` whiteboard profile. During development, at least 10 logo/image inputs were explored informally. The report shows four whiteboard examples overall, while the repository includes three bundled sample images so the measured results below can be reproduced. The values below were recorded from the preview results shown in the application UI. `sample1.jpg` and `sample2.png` used the unmodified `balanced` preset. `sample3.png` required light tuning from that preset to obtain a usable preview. ETA values are listed in total seconds even though the UI displays them in `mm:ss` format.

The table reports continuous pen-down strokes, retained path points, generated URScript motion lines, estimated path length, and planner ETA. Script lines are approximately `planned points + 2 x planned strokes` because each stroke adds an approach move and a lift move. The ETA values should be interpreted as planner-side estimates for comparing job complexity, not exact completion times. In simulator use, complex or heavily tuned drawings could take longer than the displayed ETA, so real runtime error remains unmeasured.

Table 11: Measured whiteboard planner results.

Sample Image	Planned Strokes	Planned Points	Script Lines	Path Length (m)	ETA (s)
sample1.jpg	10	146	166	2.03	76
sample2.png	19	232	270	4.93	176
sample3.png	4	173	181	4.44	156

8.10 Whiteboard Tuning Result

Figure 11 isolates `sample3.png`, which required manual tuning. With the default `balanced` preset, the preview produced 88 planned strokes, 269 planned points, and 445 script lines. After reducing `approxEpsilonFrac` and `simplifyEpsilon` to their minimum values and setting `blurKSize=10` in `all_binary` mode, the preview was reduced to 4 planned strokes, 173 planned points, and 181 script lines. The tuned preview followed the major outline more cleanly, reducing planned strokes by 84 and script lines by 264 before execution.

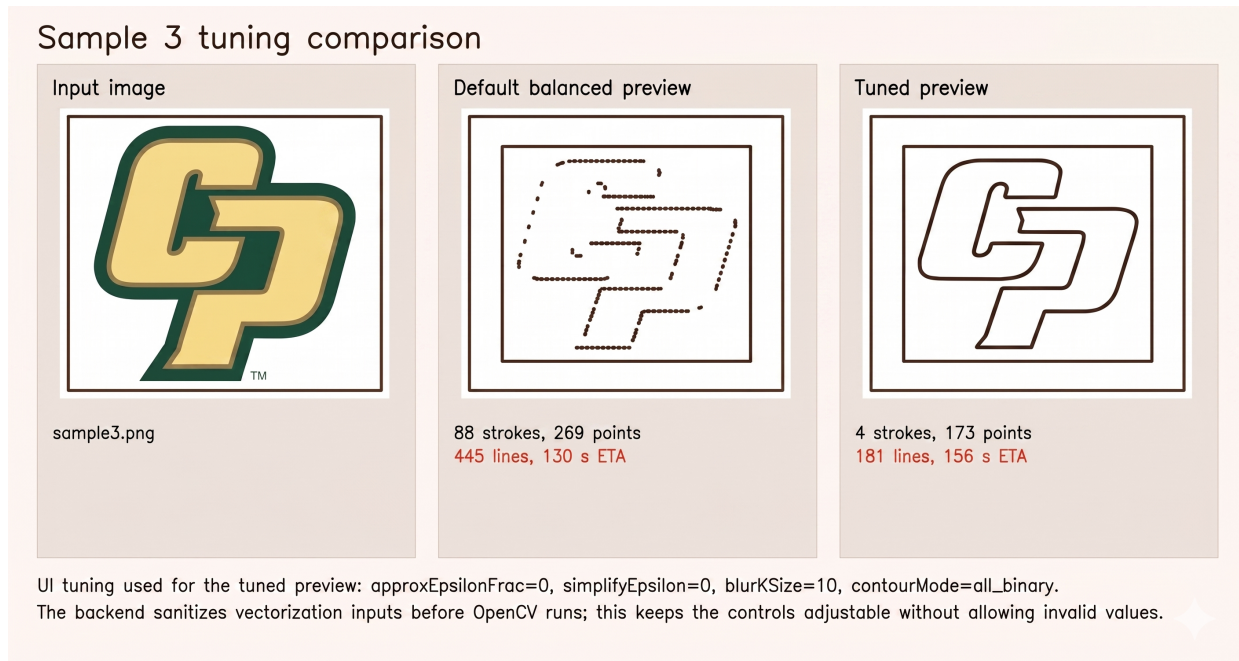


Figure 11: Manual tuning result for `sample3.png`.

8.11 Evaluation Against Criteria

Table 12: Evaluation against criteria.

Evaluation Criterion	Outcome	Evidence
Accuracy	simulator-supported	board and tic-tac-toe targets resolved to intended regions
Repeatability	qualitative evidence	repeated runs followed the same generated paths for the same profiles
Status fidelity	observed improvement	RTDE tracked motion state better than fixed timing alone
Safety compliance	planner checks	draw jobs report stroke count, point count, path length, and script size
Usability	browser workflow exercised	all three modes operated from one dashboard without manual backend calls

8.12 Results Analysis

Beyond checking whether each workflow could run, the validation process clarified how the system behaved as an integrated application. The strongest part of the project was the direct mapping path used by the board workflows. Chess movement, chess pick/place sequencing, and tic-tac-toe marking were easier to validate because the user-selected square or cell could be traced through the same sequence each time: UI selection, backend route, calibration profile, generated URScript, URSim response, and UI status update. This made errors easier to isolate because a wrong result usually pointed to either the selected profile, the calibrated anchors, or the generated pose.

Integrating chess, tic-tac-toe, and whiteboard drawing into one architecture showed why the shared layers were useful. Control ownership, calibration profiles, status polling, and URScript dispatch could be reused across all modes, while each task still kept its own planning logic. Chess used discrete square movement and suction state, tic-tac-toe converted a selected cell into bounded mark strokes, and whiteboard mode required image vectorization, tuning, preview metrics, and safety checks. This confirmed the value of the layered, pipeline-based design: common infrastructure reduced duplication, while isolated task modules kept mode-specific complexity from spreading through the rest of the system.

The most important lesson was that the whiteboard workflow is less predictable because it depends on image content before robot planning even begins. During testing, clean high-contrast images produced reasonable previews quickly, while noisy or logo-like inputs required

parameter tuning before the planned path looked usable. The most surprising result was that a visually simple output could still have a long planned path: after tuning, `sample3.png` had only 4 planned strokes but still required 173 points and a 4.44 m path. That changed how the project interpreted validation metrics. Stroke count alone was not enough; point count, path length, script size, visual preview quality, and estimated time all had to be reviewed together before execution.

The measured whiteboard results also showed how quickly complexity changes across inputs. `sample2.png` produced 19 strokes, 232 points, a 4.93 m planned path, and a 176 s ETA, compared with `sample1.jpg` at 10 strokes, 146 points, a 2.03 m path, and a 76 s ETA. These values helped make the validation more concrete because they connected preview appearance to measurable planning outputs.

Overall, the system met its main simulation objective: it translated user tasks into executable UR3e behavior through one web interface. The strongest evidence is integrated workflow execution, preview metrics, safety checks, and UI status reporting. The remaining hardware gaps are summarized in Tables 9 and 10.

9 Limitations and Challenges

The largest technical challenge was calibration. The system depends on manually defined anchors, so even small setup errors can create visible drift at other target locations. This was especially important for board alignment and whiteboard drawing, where small pose differences are easy to notice.

A second limitation is the simulator-only evaluation. URSim was sufficient for developing the communication path, status logic, and drawing pipeline, but it does not capture physical variation in tool mounting, pen contact, suction behavior, or setup repeatability.

For real hardware, the main safety concerns would be calibration drift, unexpected motion, and tool contact. If the physical board or whiteboard shifts after calibration, a target that appears safe in the UI could map to the wrong physical location. Unexpected motion could also occur if a generated pose is reachable in simulation but awkward for the real arm because of joint limits, tool mounting, or nearby obstacles. Tool contact is especially important for suction and pen-based drawing: too little contact may fail to pick up a piece or mark the board, while too much contact may damage the tool, board surface, or robot setup. A physical deployment should therefore begin with low-speed dry runs above the surface, workspace clearance checks, emergency-stop readiness, and small calibration verification moves before allowing pick/place or pen-down commands.

The whiteboard mode also depends heavily on input image quality. Clean, high-contrast images produce usable previews, while noisy or complex inputs can create weak or overly

dense contour plans. ETA is similarly limited: it is useful for comparing jobs, but it does not yet model every factor that affects real execution time.

Finally, the software scope was intentionally constrained. Raster image input is supported, but direct SVG or text workflows were not completed in the final version. This was a deliberate tradeoff to keep the implemented pipeline stable and reviewable rather than expanding into too many partially finished features.

Figure 12 shows a separate input-quality comparison in whiteboard mode. The clean-input preview produced 19 strokes and 2305 points after additional tuning, while the complex-input preview still produced 381 strokes and 4483 points. These counts are not part of the measured sample set in Table 11; they are included only to show how image quality and tuning choices can sharply change preview complexity.

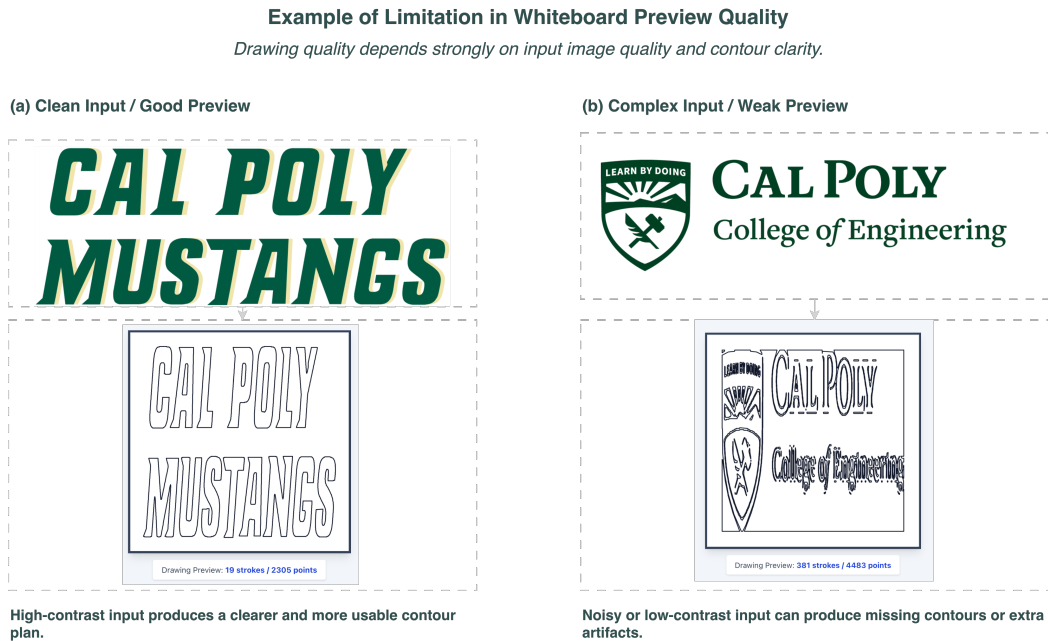


Figure 12: Whiteboard input-quality limitation.

10 Future Work

- Immediate next step: validate the full pipeline on the physical UR3e by recalibrating anchors, measuring placement and marking repeatability, and comparing real timing against simulator behavior.
- Near-term software improvements: improve contour fidelity for complex images, optimize stroke ordering, and recalibrate ETA estimates from measured runs.
- Longer-term extensions: add SVG or text-based drawing workflows, fill or hatching support, and camera-based closed-loop correction for automatic error compensation.

11 Conclusion

This project developed a web control platform that converts operator inputs into calibrated UR3e behavior. The final system integrates a React and TypeScript frontend, Express route handlers, calibration modules, URScript generation, RTDE-assisted status feedback, and an OpenCV-based drawing pipeline for chess-style movement, tic-tac-toe marking, and whiteboard drawing.

The project demonstrates that a single browser interface can support discrete board movement, bounded marking, and image-driven drawing without exposing the user to raw robot commands. The implemented platform provides end-to-end URSim operation, drawing preview and safety checks, control locking, and live moving/idle feedback. The primary limitation is that physical UR3e validation remains future work. Placement accuracy, suction pickup and release reliability, pen contact behavior, and final drawing quality were not measured on hardware, so the results should be interpreted as URSim-validated software behavior rather than confirmed physical task performance.

12 Source Code Availability

The source code for this project is available at: https://github.com/tkonchok/UR3e_Web_Control.git

13 Schedule and Development Milestones

The project developed iteratively across the two-quarter senior project sequence. The timing below is retrospective and grouped by the major development stages.

13.1 First Quarter

Table 13: First quarter schedule.

Time Period	Main Focus	Key Deliverables
Weeks 1–4	Proposal development and project direction	Defined the initial chess interaction idea, researched tools, and shaped the proposal
Weeks 5–7	Scope refinement and feasibility planning	Revised the proposal, simplified scope, and focused on a browser-based UR3e workflow
Weeks 8–10	Final proposal preparation	Finalized the proposal direction, confirmed the URSim workflow, and prepared to transition into implementation

13.2 Second Quarter

Table 14: Second quarter schedule.

Time Period	Main Focus	Key Deliverables
Weeks 1–2	Proposal finalization and implementation start	Finalized approval and began implementation with simplified scope
Weeks 3–4	First board interaction demo and early calibration	Added grid interaction, target selection, robot movement, dry-run mode, control locking, early tic-tac-toe logic, and board anchors
Weeks 4–5	Calibration refinement and chess workflow expansion	Improved move-to-square behavior and added chess-piece state tracking for pick/place
Weeks 5–6	Robot-side tic-tac-toe marking and UI integration	Added pen-based X/O drawing and unified chess/tic-tac-toe dashboard behavior
Weeks 7–8	Structured board interaction expansion	Refined board interactions and integrated board workflows into one interface
Weeks 8–9	Chess and whiteboard feature expansion	Connected chess pick/place, image upload, contour extraction, preview, drawing execution, and multi-mode UI
Week 10 and Finals Week	Refinement, live status, and final report	Added RTDE live status feedback, cleaned the repository, improved documentation, and completed the report

13.3 Milestones

- M1: Project idea narrowed to chess-related UR3e interaction through a web interface
- M2: Proposal finalized with a simplified and realistic scope
- M3: URSim development workflow and basic command communication established
- M4: First board interaction demo completed with target-square selection and robot movement
- M5: Dry-run mode and control locking integrated into the early workflow
- M6: First version of the three predefined manual anchor points implemented for board calibration
- M7: Chess piece state integrated into the UI for pick/place tracking

- M8: Tic-tac-toe robot-side marking implemented with pen-based X/O drawing
- M9: Whiteboard image upload, contour preview, and drawing execution implemented
- M10: RTDE live robot status integrated into the UI
- M11: Final repository cleanup, documentation, and report completed

References

- [1] M. A. Goodrich and A. C. Schultz. “Human-Robot Interaction: A Survey.” *Foundations and Trends in Human-Computer Interaction*, 1(3):203–275, 2007. <https://doi.org/10.1561/11000000005>
- [2] P. F. Hokayem and M. W. Spong. “Bilateral Teleoperation: An Historical Survey.” *Automatica*, 42(12):2035–2057, 2006. <https://doi.org/10.1016/j.automatica.2006.06.027>
- [3] F. Melendez-Fernandez, C. Galindo, and J. Gonzalez-Jimenez. “A Web-Based Solution for Robotic Telepresence.” *International Journal of Advanced Robotic Systems*, 2017. <https://doi.org/10.1177/1729881417743738>
- [4] R. Y. Tsai and R. K. Lenz. “A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration.” *IEEE Transactions on Robotics and Automation*, 5(3):345–358, 1989. <https://doi.org/10.1109/70.34770>
- [5] I. Enebuse, M. Foo, B. K. K. S. M. Ibrahim, H. Ahmed, F. Supmak, and S. A. Odongo. “A Comparative Review of Hand-Eye Calibration Techniques for Vision Guided Robots.” *IEEE Access*, 9:113143–113155, 2021. <https://doi.org/10.1109/ACCESS.2021.3104514>
- [6] A. Mohammed, L. Wang, and R. X. Gao. “Integrated Image Processing and Path Planning for Robotic Sketching.” *Procedia CIRP*, 12:434–439, 2013. <https://doi.org/10.1016/j.procir.2013.09.035>
- [7] Universal Robots. “Develop with URScript.” <https://www.universal-robots.com/developer/urscript/>
- [8] Universal Robots. “Script Manual, e-Series.” <https://www.universal-robots.com/download/manuals-e-seriesur-series/script/script-manual-e-series-sw-513/>
- [9] Universal Robots. “Real-Time Data Exchange (RTDE).” <https://www.universal-robots.com/developer/communication-protocol/rtde/>

- [10] Universal Robots. “Real-Time Data Exchange (RTDE) Guide.” <https://docs.universal-robots.com/tutorials/communication-protocol-tutorials/rtde-guide.html>
- [11] Universal Robots. “Dashboard Server Remote Control Interface e-Series.” https://www.universal-robots.com/manuals/EN/HTML/SW5_24/Content/prod-dashboard/Dashboard_table.htm
- [12] Universal Robots. “Overview of Client Interfaces.” <https://www.universal-robots.com/articles/ur/interface-communication/overview-of-client-interfaces/>
- [13] OpenCV. “OpenCV 4.x Documentation.” <https://docs.opencv.org/4.x/>
- [14] NumPy Developers. “NumPy Documentation.” <https://numpy.org/doc/>
- [15] Express.js. “Express Documentation.” <https://expressjs.com/>
- [16] React. “React Documentation.” <https://react.dev/>
- [17] Vite. “Vite Documentation.” <https://vite.dev/>
- [18] Node.js. “Net Module Documentation.” <https://nodejs.org/api/net.html>